

Roadmap to Success

Companion to STORY.md and ROLES-AND-BILLING-PLAN.md. Written as if this is going to be a real, paying-customer product. Each phase carries locked decisions, a checklist, and what's deliberately deferred — meant to be built as written, not re-debated mid-way.

THE HONEST CURRENT STATE

Not the optimistic version.

- A working static catalog plus a real DB/auth layer — sign-up, sign-in, workspaces, invites, all working end-to-end.
- **Exactly one real organization exists**, created by a direct database insert — no repeatable path exists for a second one.
- **Production is live but broken** — / and every /api/auth/* call were 500ing, root cause not yet found.
- **Zero tests, zero CI** anywhere in the repo — verified, not assumed.
- Zero billing — nothing charges anyone anything today.
- The roles/plan model's shape is decided on paper — Plan B/C's numbers and the 2 non-owner role levels are still TBD — and **zero of it is enforced in code** either way, beyond better-auth's built-in defaults.
- No "create category" UI, and no create/edit UI for manuals or snippets, for any role.

PRINCIPLES — CORRECTIONS, NOT JUST A CONTINUATION

Each phase's locked decisions trace back to one of these.

- 1 Stop creating state by hand**
The org, its categories, and (until recently) its owner membership were all created by direct inserts or one-off scripts — that exact pattern broke sidebar drag-and-drop.
- 2 Build the permission/plan-limit layer before more features**
Every feature shipped so far went in with no thought to who's allowed to do it — backwards for a tiered, paid-plan product.
- 3 Billing earlier than feels comfortable**
"Paying creates an organization" is the foundation the whole design sits on. Ugly-and-working beats beautiful-with-no-way-to-charge.
- 4 Fix what's live before designing what's next**
An unresolved production outage sits underneath this entire plan — Phase 0 exists because of it.
- 5 Bring in a safety net exactly when the stakes go up**
Fine to have none for a personal tool. Stops being fine once plans, seats, and permissions gate what real people pay for.

THE PHASES AT A GLANCE

#	PHASE	EXIT CONDITION
0	Stabilize YOU ARE HERE	Production serves every route without a 500; broken builds can't merge silently
1	Foundations	No org/category/membership created outside the app's own code paths
2	Billing	A second real, paying org can exist without a developer touching the database
3	Core product completeness	A workspace's content is fully self-service
4	Team features	A member's capabilities are governed by their role, not just membership
5	Polish — original UX work	Sidebar UX done right, with real architecture underneath
6	Scale readiness	Ready to hold up under real, paying usage

Stabilize

YOU ARE HERE

Fix what's actually live before doing anything else. No architecture work starts until production is honest again.

LOCKED DECISIONS

- CI runs on every push to any branch, not just PRs into main.
- CI is GitHub Actions — already hosting the repo, no new tool to adopt.
- The 500 gets root-caused from the actual Vercel runtime log, not guessed at from client-side symptoms.

CHECKLIST

- ☐ Pull the real Vercel runtime log for a failing `/api/auth/get-session` request
- ☐ Identify the actual cause (bad `DATABASE_URL`, missing `BETTER_AUTH_SECRET`, or other)
- ☐ Fix it, confirm `/` and every `/api/auth/*` route returns 200
- ☐ Add `.github/workflows/ci.yml` running lint, typecheck, build
- ☐ Commit `.env.example`

DEFERRED, ON PURPOSE

Full test suite and error monitoring — both explicitly Phase 6.

EXIT CONDITION

Production serves every route without a 500; a broken build can't merge silently.

PHASE 1

Foundations

The architecture layer. Nothing in Phase 2 onward gets built until this exists.

LOCKED DECISIONS

- **Plan limits live in one config file** (`lib/config/plan-limits.ts`) — every plan-gated feature reads from it, never hardcodes a limit inline.
- **Permission checks go through one reusable helper** (`requireOrgRole()`), called at the top of every org-scoped server action.
- **Category creation gets a real path** — a form + server action. Manual DB seeding is retired for good.

⚠ Resolving the "2 TBD role levels": reuse better-auth's `owner/admin/member` rather than building a parallel role system. Organization Manager = `owner`.

CONFIRM BEFORE BUILDING

- **A seat = any active member including the owner** — the owner occupies seat 1, not a free extra slot on top of the plan's count.
- `organization:delete` is removed from the `owner` role and reserved for the platform superadmin — implements the rule already locked in `ROLES-AND-BILLING-PLAN.md` #1, which nothing had built yet.

CHECKLIST

- ☐ Lock Plan B/C's actual seat-pricing numbers — currently TBD
- ☐ Create `lib/config/plan-limits.ts` for plans A/B/C
- ☐ Build `getSeatUsage(orgId)`, owner included
- ☐ Enforce the seat cap in the invite flow
- ☐ Build `requireOrgRole()` helper
- ☐ Retrofit `reorderCategoryAction` to call it
- ☐ Override better-auth's org access control to remove `organization:delete` from `owner`
- ☐ Build `createCategoryAction` + a minimal add-category form
- ☐ Document the role mapping in `ROLES-AND-BILLING-PLAN.md` once confirmed

DEFERRED, ON PURPOSE

Icon picker / background theme UI polish — Phase 3. Billing itself — Phase 2.

EXIT CONDITION

No org, category, or membership row for any future workspace is ever created outside the app's own code paths.

PHASE 2

Billing

The foundation the whole roles/plan design sits on. Nothing here is optional if this is meant to be a real product.

LOCKED DECISIONS

⚠ **Processor: Stripe** — most mature Next.js ecosystem support, native per-seat pricing.

CONFIRM BEFORE BUILDING

- Stripe Checkout (hosted), not a custom payment form — PCI compliance handled entirely by Stripe.
- Stripe subscriptions with quantity = seat count, mapped directly onto the per-seat curves in `ROLES-AND-BILLING-PLAN.md`.
- `organization.plan` becomes the live trigger for Phase 1's plan-limits config, updated by webhook.
- A downgrade that would put the org over the new plan's seat limit gets **blocked**, not auto-applied — members have to be removed down to the new limit first.

CHECKLIST

- ☐ Create Stripe account, get test-mode API keys
- ☐ Add Stripe env vars to `.env.example`, `.env.local`, Vercel
- ☐ Define 3 Stripe Products (A/B/C) with tiered/graduated seat pricing
- ☐ Rebuild /onboarding: name → pick plan → Stripe Checkout
- ☐ Build the webhook handler for checkout completed / subscription updated / cancelled
- ☐ Update `organization.plan` and seat count from webhook events
- ☐ Handle failed payment with at minimum a visible banner/lock state

DEFERRED, ON PURPOSE

Annual billing, coupons/discounts, proration edge cases beyond Stripe's own handling, manual/enterprise invoicing.

EXIT CONDITION

A second real, paying organization can come into existence without a developer touching the database.

Core Product Completeness

No create/edit UI for manuals or snippets exists for any role today, and public pages ignore the database. This phase closes both gaps.

LOCKED DECISIONS

- One shared editor for manuals and snippets — they already share the same table shape.
- Public pages switch to "DB first, static fallback," matching the pattern subpages already use.
- Delete is soft (a status field), not a row removal — reversible by design.

CHECKLIST

- ☐ Build the manual/snippet create form
- ☐ Build edit — same form, pre-filled
- ☐ Build soft-delete
- ☐ Update / and / [category] to query DB first, static fallback only when empty
- ☐ Icon picker — any lucide-react icon
- ☐ Background theme presets — a small fixed set

DEFERRED, ON PURPOSE

Rich text/WYSIWYG editing — start with plain textarea + markdown. Custom background image uploads — presets only for now.

EXIT CONDITION

A workspace's content is fully self-service — no static file or seed script needed for real content.

Team Features

Makes the role model from Phase 1 mean something day-to-day, not just at invite/delete-org time.

LOCKED DECISIONS

- Member-edit-quota enforcement happens in the same create/edit actions from Phase 3 — checked before write, incremented on success.
- Seat management UI extends `/workspace/members` (already exists), no new page.

CHECKLIST

- ☐ Wire `memberEditQuota` reads/writes into the Phase 3 editor actions
- ☐ Add a remaining-quota indicator in the editor UI
- ☐ Extend `/workspace/members` with seat count vs. limit, and `remove-member`
- ☐ Confirm the Phase 1 role mapping is fully enforced here

DEFERRED, ON PURPOSE

Per-member custom permission overrides beyond the 3 fixed levels — intentionally not planned.

EXIT CONDITION

A member's capabilities are governed by their role, not just whether they're in the org.

PHASE 5

Polish — the Original UX Work

The sidebar drag-and-drop/scroll work that started this whole conversation, revisited with real architecture underneath.

LOCKED DECISIONS

→ This phase is UX-only. No new data model or permission changes — a deliberate guardrail against scope creep back into architecture work.

CHECKLIST

- ☐ Revisit sidebar category UX with real category creation in place
- ☐ General visual pass — spacing, empty states, loading states
- ☐ Mobile responsiveness pass

DEFERRED, ON PURPOSE

Nothing new gets added in this phase — it's "make what exists good," not "add more."

EXIT CONDITION

The UX work that started this whole conversation, done with the architecture actually supporting it.

Scale Readiness

Coverage grows exactly where the stakes are highest — money and access control — not as blanket coverage for its own sake.

LOCKED DECISIONS

⚠ **Test framework: Vitest** — fast, clean Next.js/TypeScript fit.

CONFIRM BEFORE BUILDING

- Test scope stays narrow at first: the permission helper, plan-limit checks, seat-cap checks, the Stripe webhook handler.
- Error monitoring: Sentry (or equivalent) — standard choice, clean Next.js integration.

CHECKLIST

- ☐ Add Vitest, test `requireOrgRole`
- ☐ Test plan-limit and seat-cap enforcement
- ☐ Test the Stripe webhook handler (mocked events)
- ☐ Wire up Sentry for client and server error capture
- ☐ Re-verify the infra-swap boundary holds

DEFERRED, ON PURPOSE

Full end-to-end test suite, load testing.

EXIT CONDITION

Ready to hold up under real, paying usage — not just "it worked when I tried it."

INFRA INDEPENDENCE

None of the above depends on staying on Neon or Vercel. The org/role/plan model, the permission-check pattern, and the billing hook points are the same regardless of what's underneath. The one discipline required: keep database access behind **lib/db** and **lib/actions**, and auth behind **lib/auth** — so a future move only touches those adapter layers, never the business logic on top.

LATER: BRINGING THIS INTO THE APP ITSELF

Once this plan is solid and not actively changing, the goal is to make it a subpage under the Manuals category in Codestash itself — browsable and searchable in-app, the same way the "next16-neon-better-auth" manual already works. Not started yet; this document is the staging ground for that content.